



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 8

Team No.:

Project Title: Linux Process Monitoring and Control System

Team Members

Roll Number	Student Name	Signature
2520030561	SURISSETTI PREM SAI	
2520030106	D. MOHIT VENKAT SAI	
2520030183	N. YESWANTH KUMAR	
2520030480	S. RIKHIL	

1. Abstract

Modern Linux systems run many concurrent processes, and users need a fast way to observe resource usage and intervene when a process misbehaves, without the overhead of a full desktop monitor. Standard tools such as `top` and `ps` read process data from the `/proc` pseudo-filesystem but do not always give an interactive, script-friendly way to control processes. This project implements a lightweight, terminal-based process monitoring and control system for Linux that reads live CPU, memory, and process-state data directly from `/proc/[pid]/stat`, `/proc/[pid]/status`, and `/proc/meminfo`, and renders it using the `ncurses` library. Beyond passive monitoring, the tool lets the user select a process and send POSIX signals (`SIGSTOP`, `SIGCONT`, `SIGTERM`, `SIGKILL`) to pause, resume, or terminate it, and reports resource usage using `getrusage()`. The system is implemented in C on Ubuntu Linux and demonstrates core Operating Systems concepts including process states, the `/proc` filesystem, signal handling, and system-level resource accounting. The expected outcome is a working command-line utility that refreshes process statistics in real time, supports sorting by CPU or memory usage, and allows safe, signal-based process control.

2. Problem Statement

Users and administrators often need to observe and control running processes on a Linux machine without relying on heavyweight GUI tools. While `/proc` exposes rich per-process information, extracting and presenting it in a readable, live-updating form -- and safely controlling misbehaving processes -- requires direct use of Linux system calls and

signal mechanisms. This project addresses the need for a lightweight, terminal-based tool that monitors process resource usage and provides direct process control using POSIX system calls, giving practical insight into process scheduling, resource accounting, and inter-process signaling in Linux.

3. Objectives

1. To parse and periodically refresh process information from the /proc filesystem (/proc/[pid]/stat, /proc/[pid]/status, /proc/meminfo).
2. To display live CPU and memory usage per process in a scrollable, sortable terminal UI built with ncurses.
3. To implement process control operations (pause, resume, terminate) using POSIX signals (SIGSTOP, SIGCONT, SIGTERM, SIGKILL).
4. To measure and report resource usage using getrusage() and reap child processes correctly with waitpid().

4. Proposed Methodology

The system will be implemented in C on Ubuntu Linux. A background loop will periodically scan /proc for active process IDs and parse each process's /proc/[pid]/stat and /proc/[pid]/status files to extract CPU time, memory usage (RSS), and process state. System-wide memory statistics will be read from /proc/meminfo. This data will be rendered in a live, auto-refreshing terminal dashboard built with the ncurses library, supporting sort-by-CPU and sort-by-memory views.

The user will be able to select a process from the list and issue a signal to it (via kill()/signal()) to pause (SIGSTOP), resume (SIGCONT), or terminate (SIGTERM/SIGKILL) it. getrusage() will report the monitoring tool's own resource consumption. Error handling will cover permission-denied cases and races where a process exits mid-scan.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
/proc filesystem parsing	Source of live per-process CPU, memory, and state data
kill() / signal()	Send SIGSTOP, SIGCONT, SIGTERM, SIGKILL to control processes
getrusage()	Measure resource usage of the monitoring tool
Process states & scheduling	Interpret and display Running/Sleeping/Zombie/Stopped states
opendir() / readdir()	Enumerate active PIDs under /proc
waitpid()	Reap terminated child processes and avoid zombies

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030561	SURISSETTI PREM SAI	Implemented /proc parsing and the core data-collection module (CPU, memory, process state extraction)
2520030106	D. MOHIT VENKAT SAI	Developed the ncurses-based terminal UI, live refresh, and sort-by-CPU/memory views
2520030183	N. YESWANTH KUMAR	Implemented signal-based process control (SIGSTOP, SIGCONT, SIGTERM, SIGKILL) and error handling
2520030480	S. RIKHIL	Handled getrusage()-based resource reporting, testing, documentation, and GitHub repository management

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Target OS providing the /proc filesystem and POSIX APIs
C / C++	Implementation language for system-level programming
ncurses	Terminal UI library for the live dashboard
GCC / Make	Compilation and build automation
Git / GitHub	Version control and repository submission

8. Expected Outcome

The project will deliver a working command-line process monitor for Linux that displays live, auto-refreshing CPU and memory statistics for all running processes, sorted by resource usage, and allows the user to pause, resume, or terminate any selected process (permissions allowing) via POSIX signals.

The tool will demonstrate correct parsing of /proc, safe signal-based process control with appropriate error handling, and will report its own resource footprint using getrusage(), giving a practical demonstration of key Operating Systems concepts covered in the course.

9. GitHub Repository

GitHub Repository Link: https://github.com/premsaisurisetty-a11y/JellyBean_OSSP

Repository Created:

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: _____

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____